TP 5 : scorer la fraude en temps réel

Travail en binôme. Plateformes : **Databricks Free Edition** (entraînement), **Google Cloud** (Cloud Run, Cloud Shell, BigQuery), le **Kafka de l'enseignant**.
Prérequis : cours de la séance 5 ; tables `silver_payments`, `orders`, `customers` et `gold_payment_features` de `workspace.shopflow` (TP 2) ; environnement Cloud Shell du TP 4 (`~/tp4`, `env.sh`, `kafka_conf.py`).

> Contexte : ShopFlow veut un premier modèle de fraude en production. Vous l'entraînez sur l'historique (2024 à octobre 2025), vous le déployez comme service, vous scorez le flux de novembre 2025 rejoué dans Kafka, et vous surveillez ce qui se passe.

Tout le code et toutes les commandes vous sont fournis. Votre travail consiste à **exécuter les étapes dans l'ordre**, à **lire les résultats** et à **les expliquer** en répondant aux questions ❓.

Vue d'ensemble : d'où vient ce TP

Ce TP est la dernière étape du fil ShopFlow. Le schéma rassemble les briques construites en cinq séances : chaque rectangle est un service avec son rôle en une ligne, sa couleur dit ce qu'il fait (calcul, stockage, messages, apprentissage), et les pastilles noires donnent le ou les TP où il sert.

TP	Ce qu'il a produit	Ce que le TP 5 en fait
1	L'entrepôt ShopFlow (BigQuery et Databricks)	Le scoreur y lit les clients ; le monitoring y lit les rétrofacturations
2	Les tables `silver_payments`, `orders`, `customers` et `gold_payment_features`	Les données d'entraînement ; la table gold sert de référence pour vérifier la librairie de features
3	Le même calcul de features sur 10 fois plus de données, avec plusieurs moteurs	Polars, le moteur de la librairie commune de features ; le choix du moteur à l'échelle (bilan, partie 5)
4	Le Kafka, `env.sh`, `kafka_conf.py` et le dataset `shopflow_rt`	Le flux à scorer, l'environnement de travail et les tables de résultats
5	Le modèle, son API, le scoreur et le monitoring	Ce que vous construisez aujourd'hui

Vue d'ensemble : d'où vient ce TP (suite)

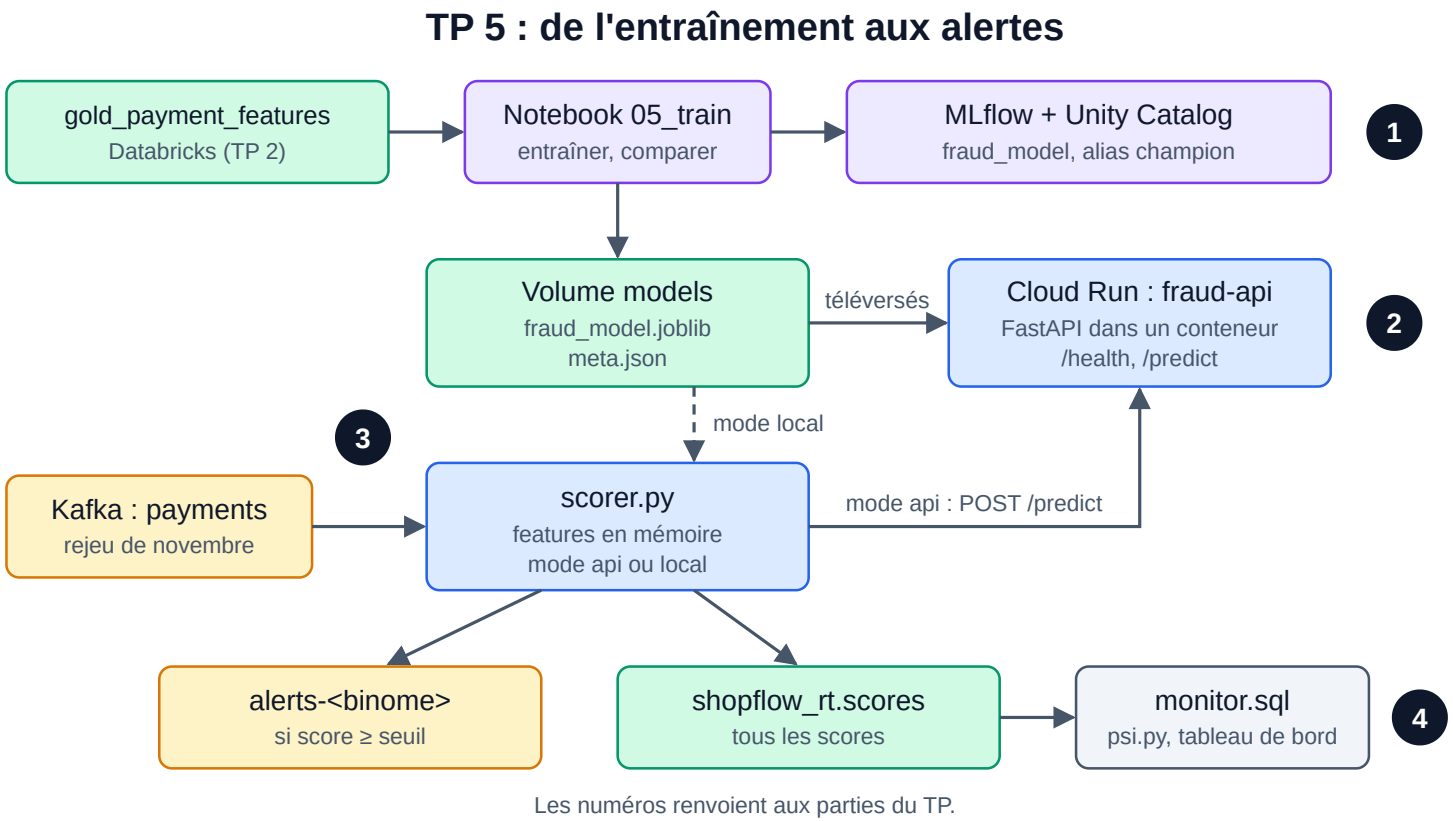


Schéma : ce que vous construisez, partie par partie ; les numéros renvoient aux parties du TP.

Un seul calcul des features, pour l'entraînement et pour le scoreur

En production, le modèle doit recevoir des features calculées **exactement** comme à l'entraînement. Sinon, il voit des valeurs qu'il n'a jamais apprises et se trompe sans aucun message d'erreur : c'est l'**écart entraînement / service** (cours, section 3.2).

> **Dans la vraie vie**, on repartirait du **feature store de Databricks** : les features seraient enregistrées dans une table de features Unity Catalog, l'entraînement les lirait, et le scoreur les interrogerait dans le magasin en ligne du feature store, avec la garantie d'avoir les mêmes valeurs des deux côtés. Ici, ce n'est pas possible : en Free Edition, ce magasin n'est pas ouvert à Internet, et notre scoreur tourne sur Google Cloud.

Un seul calcul des features, pour l'entraînement et pour le scoreur (suite)

À la place, le TP utilise une **librairie commune**, `features_lib.py`, écrite avec **Polars** (séance 3) pour passer à l'échelle :

- **à l'entraînement** (Databricks), elle calcule les features de tout l'historique des paiements ;
- **dans le scoreur** (Cloud Shell), elle calcule les features de chaque petit lot de paiements reçus de Kafka, à partir de l'historique que le scoreur garde en mémoire.

C'est **le même fichier** des deux côtés. Son empreinte (SHA-256, une sorte de signature du contenu) est enregistrée avec le modèle, et le scoreur refuse de démarrer si son fichier est différent.

Partie 1 : entraîner, suivre, enregistrer (Databricks)

1.0 Créer le notebook et la librairie

1. Dans Databricks, menu de gauche **Workspace**, puis **Create > Notebook**.
2. Renommez-le `05_train` (cliquez sur le titre en haut). Langage : **Python**.
3. En haut à droite, menu **Connect** : choisissez **Serverless**.
4. **La librairie commune.** Revenez dans **Workspace**, ouvrez le dossier qui contient `05_train`, puis **Create > File**. Renommez le fichier `features_lib.py` : double-cliquez sur son titre dans l'onglet, puis remplacez le nom proposé. Collez-y le code ci-dessous, il s'enregistre tout seul. Il doit être **dans le même dossier que le notebook** pour que `import features_lib` le trouve. Ne le modifiez plus ensuite : son empreinte est vérifiée par le scoreur (partie 3).
5. Dans le notebook, pour chaque bloc de code qui suit : créez une nouvelle cellule (`+ Code` sous la précédente), collez le bloc, puis exécutez-la avec `Maj+Entrée`. Attendez la fin d'une cellule avant de lancer la suivante.

`features_lib.py` (le code des features ; ce sont les définitions de `gold_payment_features` du TP 2, traduites en Polars) :

1.0 Créer le notebook et la librairie (suite)

```
"""Features de fraude ShopFlow : un seul code pour l'entraînement (Databricks) et le scoreur (Cloud Shell).
Mêmes définitions que la table gold_payment_features du TP 2."""
import polars as pl

FEATURES = ["amount", "card_nb_tx_1d", "card_nb_tx_7d", "card_avg_amount_7d", "card_avg_amount_30d",
            "amount_ratio_30d", "ip_nb_tx_1h", "ip_nb_cards_1h", "account_age_days",
            "ship_country_mismatch", "is_guest"]

# colonnes attendues en entrée : une ligne par paiement, refusés compris ; heures en UTC
SCHEMA = {"payment_id": pl.Int64, "payment_ts": pl.Datetime("us"), "amount": pl.Float64,
          "ip": pl.String, "customer_id": pl.Int64, "card_id": pl.Int64,
          "shipping_country": pl.String, "customer_country": pl.String, "signup_date": pl.Date}


def _par_carte(fn: str, col: str, duree: str) -> pl.Expr:
    """Fenêtre [t - durée, t] sur les paiements de la même carte, comme
    « RANGE BETWEEN ... PRECEDING AND CURRENT ROW » en SQL : le paiement courant est inclus."""
    return getattr(pl.col(col), fn)("payment_ts", window_size=duree, closed="both").over("card_key")


def _cartes_par_ip_1h(df: pl.DataFrame) -> pl.DataFrame:
    """Nombre de cartes distinctes vues sur la même IP dans l'heure [t - 1 h, t]."""
```

1.0 Créer le notebook et la librairie (suite)

```
    df = df.sort("ip", "payment_ts")
    r = (df.rolling(index_column="payment_ts", period="1h", group_by="ip", closed="both")
           .agg(pl.col("card_key").n_unique().alias("ip_nb_cards_1h"))
           .sort("ip", "payment_ts", maintain_order=True))   # Polars rend les IP dans un ordre quelconque
    # garde-fou : une ligne par paiement, dans le même ordre que df
    assert r.height == df.height and (r["payment_ts"] == df["payment_ts"]).all()
    return df.select("payment_id").with_columns(r["ip_nb_cards_1h"])


def add_features(payments: pl.DataFrame) -> pl.DataFrame:
    """Ajoute les 11 colonnes de FEATURES à un DataFrame de paiements."""
    if getattr(payments.schema["payment_ts"], "time_zone", None):     # heure avec fuseau : UTC sans fuseau
        payments = payments.with_columns(
            pl.col("payment_ts").dt.convert_time_zone("UTC").dt.replace_time_zone(None))
    df = payments.with_columns([pl.col(c).cast(t) for c, t in SCHEMA.items()])
    # paiement sans carte (PayPal) : une clé propre à ce paiement
    df = df.with_columns(card_key=pl.coalesce("card_id", -pl.col("payment_id")), one=pl.lit(1.0))
    df = df.with_columns(
        card_nb_tx_1d=_par_carte("rolling_sum_by", "one", "1d"),
        card_nb_tx_7d=_par_carte("rolling_sum_by", "one", "7d"),
        card_avg_amount_7d=_par_carte("rolling_mean_by", "amount", "7d"),
        card_avg_amount_30d=_par_carte("rolling_mean_by", "amount", "30d"),
```

1.0 Créer le notebook et la librairie (suite)

```
        ip_nb_tx_1h=pl.col("one").rolling_sum_by("payment_ts", window_size="1h", closed="both").over("ip"),
    )
    df = df.join(_cartes_par_ip_1h(df), on="payment_id", how="left")
    df = df.with_columns(
        amount_ratio_30d=pl.col("amount") / pl.col("card_avg_amount_30d"),
        account_age_days=pl.when(pl.col("customer_id") == 0).then(-1)
                           .otherwise((pl.col("payment_ts").dt.date() - pl.col("signup_date")).dt.total_days()),
        ship_country_mismatch=(pl.col("shipping_country") != pl.col("customer_country")).fill_null(False),
        is_guest=pl.col("customer_id") == 0,
    )
    return df.with_columns(pl.col(FEATURES).cast(pl.Float64)).drop("one")
```

Cellule 1. Installer MLflow à jour et Polars (le notebook redémarre Python, c'est normal) :

```
%pip install -q "mlflow>=3" polars
dbutils.library.restartPython()
```

1.A Les données

Cellule 2. Lire les paiements du TP 2, calculer les features avec la librairie, découper dans le temps.
Les features sont calculées sur **tous** les paiements, refusés compris ; on ne garde les paiements autorisés qu'ensuite, pour l'entraînement :

1.A Les données (suite)

```
import hashlib, json, os, platform, shutil, time
import joblib
import numpy as np
import pandas as pd
import polars as pl
import sklearn
import mlflow
from mlflow.models import infer_signature
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, roc_auc_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

import features_lib as fl
FEATURES = fl.FEATURES

spark.conf.set("spark.sql.session.timeZone", "UTC")
raw = spark.sql("""
    SELECT p.payment_id, p.payment_ts, p.amount, p.ip, p.customer_id, p.card_id, p.status, p.has_chargeback,
           o.shipping_country, c.country AS customer_country, c.signup_date
    FROM workspace.shopflow.silver_payments AS p
```

1.A Les données (suite)

```
    LEFT JOIN workspace.shopflow.orders AS o ON o.order_id = p.order_id
    LEFT JOIN workspace.shopflow.customers AS c ON c.customer_id = p.customer_id
""").toPandas()

t0 = time.time()
feats = fl.add_features(pl.from_pandas(raw))                # le même code que le scoreur
print(f"{feats.height} paiements, features calculées par Polars en {time.time() - t0:.1f} s")

df = (feats.filter(pl.col("status") == "authorized")        # paiements autorisés seulement
           .select("payment_id", "payment_ts", *FEATURES, "has_chargeback")
           .to_pandas())
df["label"] = df["has_chargeback"].fillna(False).astype(int)  # 1 = fraude (rétrofacturation), 0 = légitime

train = df[df.payment_ts < "2025-07-01"]
test = df[(df.payment_ts >= "2025-08-01") & (df.payment_ts < "2025-11-01")]
print(f"train : {len(train)} paiements, {train.label.mean():.3%} de fraudes")
print(f"test  : {len(test)} paiements, {test.label.mean():.3%} de fraudes")
```

Notez les lignes affichées.

1.A Les données (suite)

Cellule 2 bis. Vérifier la librairie contre la table gold du TP 2. Pour chaque feature, on compare les valeurs calculées par Polars à celles de `gold_payment_features`, paiement par paiement :

```
gold = (spark.table("workspace.shopflow.gold_payment_features")
        .select("payment_id", *FEATURES).toPandas())
cmp = df[["payment_id", *FEATURES]].merge(gold, on="payment_id", suffixes=("_polars", "_gold"))
ecart = {f: (cmp[f + "_polars"] - cmp[f + "_gold"].astype("float64")).abs() for f in FEATURES}
print(len(cmp), "paiements comparés")
pd.DataFrame({"écart max": {f: e.max() for f, e in ecart.items()},
              "lignes différentes": {f: int((e > 1e-6).sum()) for f, e in ecart.items()}})
```

1.A Les données (suite)

Découpage temporel du TP 5

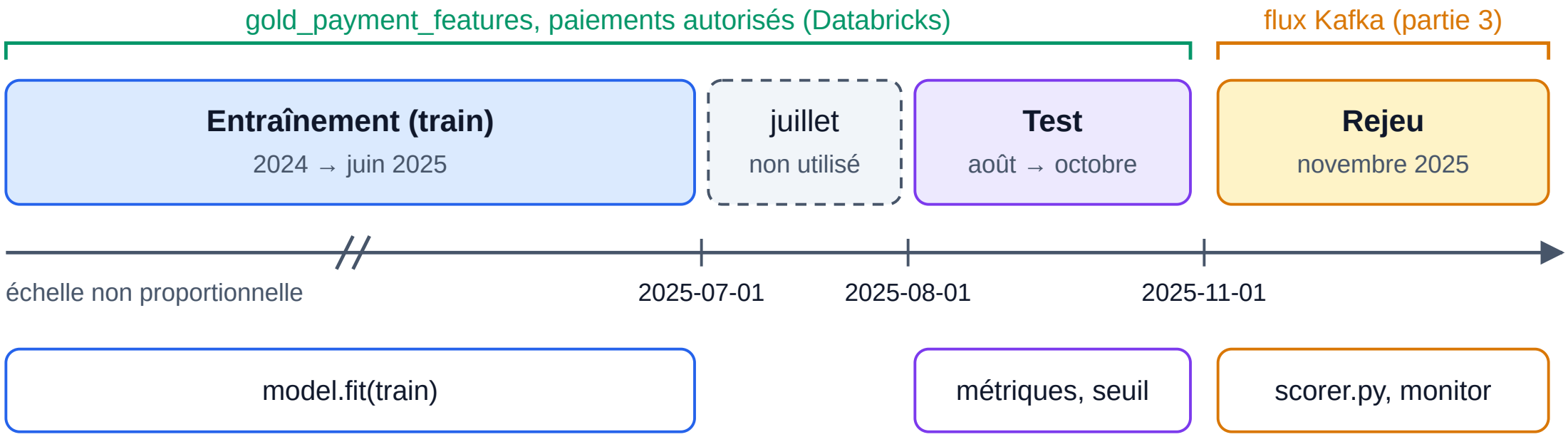


Schéma : les bornes de `train` et `test` dans le code ci-dessus, et le mois de novembre réservé au rejeu Kafka.

1.A Les données (suite)

❓ Q1. Pourquoi ne garder que les paiements autorisés ? Pourquoi un mois vide (juillet 2025) entre l'entraînement et le test ? Pourquoi s'arrêter fin octobre ?

❓ Q1 bis. La librairie Polars donne-t-elle les mêmes valeurs que la table gold du TP 2 (cellule 2 bis) ? Si des écarts apparaissent, sur quelles features ? Pourquoi vaut-il mieux un seul code de calcul des features, utilisé à l'entraînement et par le scoreur, plutôt que deux codes qui « font la même chose » ?

1.B Trois modèles suivis dans MLflow

Cellule 3. La fonction d'entraînement. Pour chaque modèle, elle ouvre une exécution (*run*) MLflow, entraîne sur `train`, mesure trois métriques sur `test`, enregistre le tout et crée une nouvelle version du modèle dans Unity Catalog :

1.B Trois modèles suivis dans MLflow (suite)

```
mlflow.set_registry_uri("databricks-uc")
MODEL_NAME = "workspace.shopflow.fraud_model"

def precision_top_k(d: pd.DataFrame, scores: np.ndarray, k: int = 10) -> float:
    """Part de vraies fraudes parmi les k paiements les mieux scorés de chaque jour."""
    t = d.assign(score=scores, day=d.payment_ts.dt.date)
    top = t.sort_values("score", ascending=False).groupby("day").head(k)
    return float(top.groupby("day").label.mean().mean())

def train_and_log(name: str, model, params: dict):
    with mlflow.start_run(run_name=name):
        model.fit(train[FEATURES], train.label)
        s = model.predict_proba(test[FEATURES])[:, 1]          # score de fraude entre 0 et 1
        metrics = {
            "auc_pr": average_precision_score(test.label, s),
            "auc_roc": roc_auc_score(test.label, s),
            "precision_top10": precision_top_k(test, s, 10),
        }
        mlflow.log_params(params)
        mlflow.log_metrics(metrics)
        info = mlflow.sklearn.log_model(
            model, name="model",
```

1.B Trois modèles suivis dans MLflow (suite)

```
            signature=infer_signature(test[FEATURES].head(100), s[:100]),
            input_example=test[FEATURES].head(3),
            registered_model_name=MODEL_NAME,
        )
        print(name, {k: round(v, 4) for k, v in metrics.items()}, "version", info.registered_model_version)
        return model, s, metrics, info
```

Cellule 4. Entraîner une régression logistique et deux réglages de gradient boosting (quelques minutes) :

1.B Trois modèles suivis dans MLflow (suite)

```
results = {}

logreg = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000, class_weight="balanced"))
results["logreg"] = train_and_log("logreg", logreg, {"model": "logreg", "class_weight": "balanced"})

hgb_params = [
    {"learning_rate": 0.1, "max_iter": 200, "class_weight": None},
    {"learning_rate": 0.05, "max_iter": 400, "class_weight": "balanced"},
]
for i, prm in enumerate(hgb_params):
    m = HistGradientBoostingClassifier(random_state=42, **prm)
    results[f"hgb_{i}"] = train_and_log(f"hgb_{i}", m, {"model": "hgb", **prm})

pd.DataFrame({k: v[2] for k, v in results.items()}).T.round(4)
```

1.B Trois modèles suivis dans MLflow (suite)

Cellule 4 bis. Comprendre les métriques. Elle donne, pour chaque gradient boosting, le nombre d'itérations réellement faites (scikit-learn arrête l'entraînement tout seul quand la perte de validation ne baisse plus) et le nombre de scores distincts. Elle calcule aussi le nombre de fraudes autorisées par jour du test, et la précision top 10 maximale qu'un modèle parfait pourrait atteindre :

```
for k in ["hgb_0", "hgb_1"]:
    m, s = results[k][0], results[k][1]
    print(f"{k} : {m.n_iter_} itérations (arrêt anticipé : {m.do_early_stopping_}), "
          f"{len(np.unique(s.round(8)))} scores distincts sur {len(s)} paiements de test")
fr = test.groupby(test.payment_ts.dt.date).label.sum()
print(f"fraudes autorisées par jour (test) : moyenne {fr.mean():.1f}, min {fr.min()}, max {fr.max()}, "
      f"jours avec moins de 10 : {(fr < 10).sum()} sur {len(fr)}")
print(f"précision top 10 maximale possible : {(fr.clip(upper=10) / 10).mean():.4f}")
```

1.B Trois modèles suivis dans MLflow (suite)

Puis, dans l'interface :

1. Cliquez sur l'icône **Experiments** (fiole) dans la barre de droite du notebook, puis sur le nom de l'expérience : vous voyez les trois exécutions. Cochez-les et cliquez sur **Compare** pour afficher leurs paramètres et métriques côte à côte.
2. Menu de gauche **Catalog > workspace > shopflow**, rubrique **Models**, puis `fraud_model` : regardez la liste des versions (une par exécution).

❓ Q2. Recopiez le tableau des métriques des trois exécutions. Lequel garder ? Pourquoi l'AUC-ROC est-elle élevée pour tous alors que l'AUC-PR est beaucoup plus basse ? Si l'un des modèles a une AUC-ROC nettement plus faible, expliquez-la avec la cellule 4 bis (nombre d'itérations, scores distincts). Quelle est la précision top 10 maximale possible (cellule 4 bis), et quelle part de ce maximum atteint le meilleur modèle ?

1.C Promouvoir et exporter

Cellule 5. Choisir le meilleur modèle (AUC-PR), lui donner l'alias `champion`, et exporter le modèle et sa fiche dans le volume `models` :

1.C Promouvoir et exporter (suite)

```
from mlflow import MlflowClient

best_name = max(results, key=lambda k: results[k][2]["auc_pr"])
best_model, best_scores, _, best_info = results[best_name]
MlflowClient().set_registered_model_alias(MODEL_NAME, "champion", best_info.registered_model_version)
print("meilleur :", best_name, "version", best_info.registered_model_version)

threshold = float(np.quantile(best_scores, 0.99))                    # 1 % des paiements en alerte
edges = np.quantile(train.amount, np.linspace(0, 1, 11)).tolist()   # déciles du montant, pour la dérive
meta = {
    "model_name": MODEL_NAME,
    "model_version": str(best_info.registered_model_version),
    "features": FEATURES,
    "threshold": threshold,
    "amount_edges": edges,
    "sklearn_version": sklearn.__version__,
    "numpy_version": np.__version__,
    "python_version": platform.python_version(),
    "polars_version": pl.__version__,
    "features_lib_sha256": hashlib.sha256(open(fl.__file__, "rb").read()).hexdigest(),
}
spark.sql("CREATE VOLUME IF NOT EXISTS workspace.shopflow.models")
```

1.C Promouvoir et exporter (suite)

```
joblib.dump(best_model, "/tmp/fraud_model.joblib")                  # écrit en local, puis copié dans le volume
shutil.copy("/tmp/fraud_model.joblib", "/Volumes/workspace/shopflow/models/fraud_model.joblib")
shutil.copy(fl.__file__, "/Volumes/workspace/shopflow/models/features_lib.py")    # la librairie voyage avec le modèle
with open("/Volumes/workspace/shopflow/models/meta.json", "w") as f:
    f.write(json.dumps(meta, indent=2))
print(json.dumps(meta, indent=2))
```

Dans *Catalog > workspace > shopflow > Models > fraud_model*, vérifiez que l'alias `champion` est affiché sur la bonne version.

Téléchargez les trois fichiers sur votre ordinateur : *Catalog > workspace > shopflow > Volumes > models*, puis, pour `fraud_model.joblib`, `meta.json` et `features_lib.py`, menu `⋮` au bout de la ligne > **Download**. Si un fichier du même nom existe déjà dans votre dossier de téléchargements, le navigateur l'enregistre sous `features_lib (1).py` : renommez-le `features_lib.py` avant l'étape 2.1.

1.C Promouvoir et exporter (suite)

❓ Q3. Ouvrez `meta.json`. Pourquoi enregistrer les versions de scikit-learn, numpy, Python et Polars, et l'empreinte de `features_lib.py`, avec le modèle ?

Partie 2 : servir le modèle avec FastAPI sur Cloud Run

2.0 Retrouver l'environnement du TP 4

Ouvrez **Cloud Shell** (bouton `>_` de la console Google Cloud) et vérifiez que l'environnement du TP 4 est toujours là :

```
ls ~/tp4/env.sh ~/tp4/kafka_conf.py ~/tp4/bin/activate
```

- **Les trois fichiers s'affichent** : passez à la commande d'entrée ci-dessous.
- **Un fichier manque** : refaites la **partie 0 du TP 4** (étapes 1, 1 bis, 2 et 3), puis revenez ici.

Commande d'entrée, à taper **dans chaque nouvel onglet Cloud Shell** (bouton `+`) de ce TP :

```
cd ~/tp4 && source bin/activate && source env.sh
```

2.0 Retrouver l'environnement du TP 4 (suite)

Ajoutez à `env.sh` le projet de l'enseignant (celui du TP 1, où se trouvent les tables partagées), puis vérifiez :

```
grep -q PROJET_ENSEIGNANT ~/tp4/env.sh || echo "export PROJET_ENSEIGNANT=esigelec-big-data-ia" >> ~/tp4/env.sh
source ~/tp4/env.sh
echo "binôme=$BINOME  projet=$PROJECT  projet enseignant=$PROJET_ENSEIGNANT"
```

Le dataset `shopflow_rt` a été créé au TP 4. Cette commande le recrée seulement s'il manque :

```
bq show $PROJECT:shopflow_rt >/dev/null 2>&1 || bq --location=EU mk --dataset $PROJECT:shopflow_rt
```

2.1 Mettre le modèle dans Cloud Shell

1. Créez le dossier de l'API :

```
mkdir -p ~/tp5/api/model
```

2.1 Mettre le modèle dans Cloud Shell (suite)

2. Dans Cloud Shell, menu `⋮` (en haut à droite du terminal) > **Importer** : choisissez les trois fichiers téléchargés (`fraud_model.joblib`, `meta.json` et `features_lib.py`). Laissez le dossier de destination proposé (`/home/<vous>/`) et attendez la fin de la barre de progression, en bas à droite.

   > **Un seul onglet Cloud Shell à la fois.** Si Cloud Shell est ouvert dans deux navigateurs, ou deux fenêtres, la session passe de l'un à l'autre (« Your session was transferred to another browser tab »). Les commandes et les imports faits dans l'onglet abandonné sont alors perdus. Si ce message s'affiche, fermez l'autre onglet, puis cliquez sur **Reconnect**.

3. Rangez le modèle et sa fiche dans le dossier du modèle, et la librairie à côté du futur scoreur, puis vérifiez. Le script cherche les fichiers dans `~` et dans `~/tp5` (au cas où vous auriez changé le dossier de destination), les range, puis compare l'empreinte de `features_lib.py` à celle qui est enregistrée dans `meta.json` :

2.1 Mettre le modèle dans Cloud Shell (suite)

```
cd ~
for f in fraud_model.joblib meta.json features_lib.py; do
  [ -f ~/$f ] || { [ -f ~/tp5/$f ] && mv ~/tp5/$f ~/; } || echo "MANQUANT : $f (refaites l'import)"
done
mv ~/fraud_model.joblib ~/meta.json ~/tp5/api/model/
mv ~/features_lib.py ~/tp4/
ls -l ~/tp5/api/model/ ~/tp4/features_lib.py && cat ~/tp5/api/model/meta.json
python3 -c "
import hashlib, json, os
meta = json.load(open(os.path.expanduser('~/tp5/api/model/meta.json')))
h = hashlib.sha256(open(os.path.expanduser('~/tp4/features_lib.py'), 'rb').read()).hexdigest()
print('features_lib.py :', 'IDENTIQUE à l entraînement' if h == meta['features_lib_sha256'] else 'DIFFÉRENT : retéléchargez-le')"
```

La dernière ligne doit afficher `IDENTIQUE à l entraînement`.

2.2 Les fichiers de l'API

Le code de l'API, `main.py`. Au démarrage, il charge le modèle et sa fiche. Il expose deux adresses : `/health` (le service répond-il ? quelle version du modèle ?) et `/predict` (reçoit les 11 features d'un paiement, renvoie le score et l'alerte) :

2.2 Les fichiers de l'API (suite)

```
cat > ~/tp5/api/main.py <<'EOF'
import json

import joblib
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel

model = joblib.load("model/fraud_model.joblib")
meta = json.load(open("model/meta.json"))
FEATURES = meta["features"]


class Payment(BaseModel):
    payment_id: int
    amount: float
    card_nb_tx_1d: float
    card_nb_tx_7d: float
    card_avg_amount_7d: float
    card_avg_amount_30d: float
    amount_ratio_30d: float
    ip_nb_tx_1h: float
```

2.2 Les fichiers de l'API (suite)

```
    ip_nb_cards_1h: float
    account_age_days: float
    ship_country_mismatch: float
    is_guest: float


app = FastAPI(title="ShopFlow fraud API")


@app.get("/health")
def health():
    return {"status": "ok", "model_version": meta["model_version"]}


@app.post("/predict")
def predict(p: Payment):
    X = pd.DataFrame([p.model_dump()])[FEATURES]
    score = float(model.predict_proba(X)[0, 1])
    return {"payment_id": p.payment_id, "score": score,
            "alert": score >= meta["threshold"], "model_version": meta["model_version"]}
EOF
```

2.2 Les fichiers de l'API (suite)

Les dépendances et l'image du conteneur. Les versions de scikit-learn, numpy et Python sont lues dans `meta.json` et recopiées automatiquement dans `requirements.txt` et dans le `Dockerfile` :

2.2 Les fichiers de l'API (suite)

```
cd ~/tp5/api
PY=$(python -c "import json; v=json.load(open('model/meta.json'))['python_version']; print('.'.join(v.split('.')[:2]))")
SK=$(python -c "import json; print(json.load(open('model/meta.json'))['sklearn_version'])")
NP=$(python -c "import json; print(json.load(open('model/meta.json'))['numpy_version'])")
echo "Python $PY   scikit-learn $SK   numpy $NP"

cat > requirements.txt <<EOF
fastapi
uvicorn[standard]
pandas
joblib
requests
scikit-learn==$SK
numpy==$NP
EOF

cat > Dockerfile <<EOF
FROM python:$PY-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

2.2 Les fichiers de l'API (suite)

```
CMD exec uvicorn main:app --host 0.0.0.0 --port \${PORT:-8080}
EOF

cat requirements.txt Dockerfile
```

Vérifiez que les versions affichées sont bien celles de `meta.json`.

2.3 Tester l'API dans Cloud Shell

On installe les mêmes versions dans l'environnement Python de Cloud Shell, plus la version de Polars de l'entraînement (elles serviront aussi au scoreur de la partie 3), on démarre l'API en arrière-plan, on l'interroge, puis on l'arrête :

```
cd ~/tp5/api
PL=$(python -c "import json; print(json.load(open('model/meta.json'))['polars_version'])")
python -m pip install -q -r requirements.txt "polars==$PL"
uvicorn main:app --port 8081 > uvicorn.log 2>&1 &
for i in $(seq 30); do curl -s localhost:8081/health > /dev/null && break; sleep 1; done   # attend que l'API réponde (30 s au plus)
curl -s localhost:8081/health ; echo
curl -s -X POST localhost:8081/predict -H "Content-Type: application/json" -d '{"payment_id": 1, "amount": 120.0,
  "card_nb_tx_1d": 1, "card_nb_tx_7d": 3, "card_avg_amount_7d": 80.0, "card_avg_amount_30d": 75.0,
  "amount_ratio_30d": 1.6, "ip_nb_tx_1h": 1, "ip_nb_cards_1h": 1, "account_age_days": 400,
  "ship_country_mismatch": 0, "is_guest": 0}' ; echo
kill %1
```

2.3 Tester l'API dans Cloud Shell (suite)

Vous devez voir `{"status":"ok", ...}` puis un score entre 0 et 1. Le premier démarrage prend une dizaine de secondes (import de scikit-learn et pandas, chargement du modèle) : la boucle `for` attend que l'API réponde. Si rien ne s'affiche, lisez `cat uvicorn.log`.

2.4 Déployer sur Cloud Run

Activer les services et donner les droits de construction (une seule fois). `gcloud run deploy --source .` fait construire l'image par Cloud Build, avec le compte de service par défaut du projet, puis la range dans Artifact Registry :

```
gcloud services enable run.googleapis.com cloudbuild.googleapis.com artifactregistry.googleapis.com
SA=$(gcloud projects describe $PROJECT --format='value(projectNumber)')-compute@developer.gserviceaccount.com
gcloud projects add-iam-policy-binding $PROJECT --member=serviceAccount:$SA --role=roles/run.builder --condition=None > /dev/null
echo "droits donnés à $SA"
```

Attendez une minute que les droits s'appliquent.

Déployer (3 à 5 minutes ; si la commande pose une question, par exemple pour créer un dépôt Artifact Registry, répondez `Y`) :

2.4 Déployer sur Cloud Run (suite)

```
cd ~/tp5/api
gcloud run deploy fraud-api --source . --region=europe-west1 \
  --allow-unauthenticated --memory=1Gi --max-instances=3
```

`--allow-unauthenticated` rend le service public : acceptable ici (données synthétiques, service supprimé en fin de séance) ; en production, on exige une identité (IAM).

Récupérer l'adresse du service et la garder dans `env.sh` :

```
export API_URL=$(gcloud run services describe fraud-api --region=europe-west1 --format='value(status.url)')
grep -q API_URL ~/tp4/env.sh || echo "export API_URL=$API_URL" >> ~/tp4/env.sh
echo $API_URL
curl -s $API_URL/health ; echo
```

2.4 Déployer sur Cloud Run (suite)

L'adresse affichée par `gcloud run deploy` (`https://fraud-api-<numéro du projet>.europe-west1.run.app`) peut différer de `$API_URL` (`https://fraud-api-<code>-ew.a.run.app`) : ce sont deux noms du même service.

2.5 Mesurer la latence

Le script `bench.py` envoie 300 fois le même paiement à l'API et mesure le temps de chaque réponse :

2.5 Mesurer la latence (suite)

```
cat > ~/tp5/bench.py <<'EOF'
import os, time
import numpy as np
import requests

url = os.environ["API_URL"] + "/predict"
body = {"payment_id": 1, "amount": 120.0, "card_nb_tx_1d": 1, "card_nb_tx_7d": 3,
        "card_avg_amount_7d": 80.0, "card_avg_amount_30d": 75.0, "amount_ratio_30d": 1.6,
        "ip_nb_tx_1h": 1, "ip_nb_cards_1h": 1, "account_age_days": 400,
        "ship_country_mismatch": 0, "is_guest": 0}
s = requests.Session()
lat = []
for i in range(300):
    t = time.perf_counter()
    r = s.post(url, json=body, timeout=30)
    r.raise_for_status()
    lat.append((time.perf_counter() - t) * 1000)
print(f"1re requête {lat[0]:.0f} ms ; ensuite p50 {np.percentile(lat[1:], 50):.0f} ms, "
      f"p99 {np.percentile(lat[1:], 99):.0f} ms")
EOF
python ~/tp5/bench.py
python ~/tp5/bench.py
```

2.5 Mesurer la latence (suite)

Notez les deux lignes. Ici, l'instance vient d'être démarrée par le déploiement et par le `curl /health` : ce n'est pas encore un démarrage à froid. **Vous relancerez ce script plus tard**, après au moins 15 minutes sans aucun appel à l'API (étape 4.0).

❓ Q4 (à compléter après l'étape 4.0). Comparez la première requête et les suivantes, puis le lancement « à chaud » et celui fait après 15 minutes sans trafic. Qu'est-ce que le démarrage à froid ? Quelle option de `gcloud run deploy` l'éviterait, et combien coûterait-elle (regardez la page de tarification de Cloud Run) ?

Partie 3 : le scoreur de flux Kafka

Le scoreur lit le topic `payments` **par petits lots** (jusqu'à 500 messages ou 1 seconde), calcule leurs features **avec** `features_lib.py`, le même code qu'à l'entraînement, score chaque paiement, publie les alertes dans votre topic `alerts-<binome>` et écrit tous les scores dans BigQuery. Il fonctionne dans deux modes :

- `api` : il envoie les features à l'API Cloud Run de la partie 2 ;
- `local` : il charge le modèle dans son propre processus.

> Le rejeu accélère le temps : 1 seconde réelle = 2 minutes de ShopFlow. Les features (« 1 jour », « 1 heure ») utilisent donc `orig_ts`, l'heure d'origine des données, et pas `event_ts`.

3.0 Vérifier que le flux arrive

À cette vitesse, les 30 jours de novembre durent 6 heures réelles : un rejeu lancé trop tôt est déjà terminé, et le scoreur n'a alors rien à lire. Le script `verif_flux.py` lit la position de fin (*high watermark*) des 6 partitions de `payments` deux fois, à 20 secondes d'intervalle, et affiche l'heure ShopFlow du dernier paiement publié :

3.0 Vérifier que le flux arrive (suite)

```
cd ~/tp4 && source bin/activate && source env.sh
cat > ~/tp4/verif_flux.py <<'EOF'
import json, time
from confluent_kafka import Consumer, TopicPartition
from kafka_conf import conf

c = Consumer(conf(**{"group.id": "verif-flux", "enable.auto.commit": False}))
parts = sorted(c.list_topics("payments", timeout=15).topics["payments"].partitions)
fin = lambda: sum(c.get_watermark_offsets(TopicPartition("payments", p), timeout=10)[1] for p in parts)
avant = fin(); time.sleep(20); apres = fin()
lo, hi = c.get_watermark_offsets(TopicPartition("payments", 0), timeout=10)
c.assign([TopicPartition("payments", 0, hi - 1)])
m = c.poll(15)
print(f"{apres - avant} paiements publiés en 20 s ; dernier orig_ts : {json.loads(m.value())['orig_ts'] if m else '?'}")
print("le rejeu tourne" if apres > avant else "LE REJEU EST ARRÊTÉ : prévenez l'enseignant")
c.close()
EOF
python verif_flux.py 2>&1 | grep -v "^%"    # masque les messages techniques de librdkafka
```

3.0 Vérifier que le flux arrive (suite)

Vous devez lire « le rejeu tourne », avec quelques dizaines de paiements en 20 s.

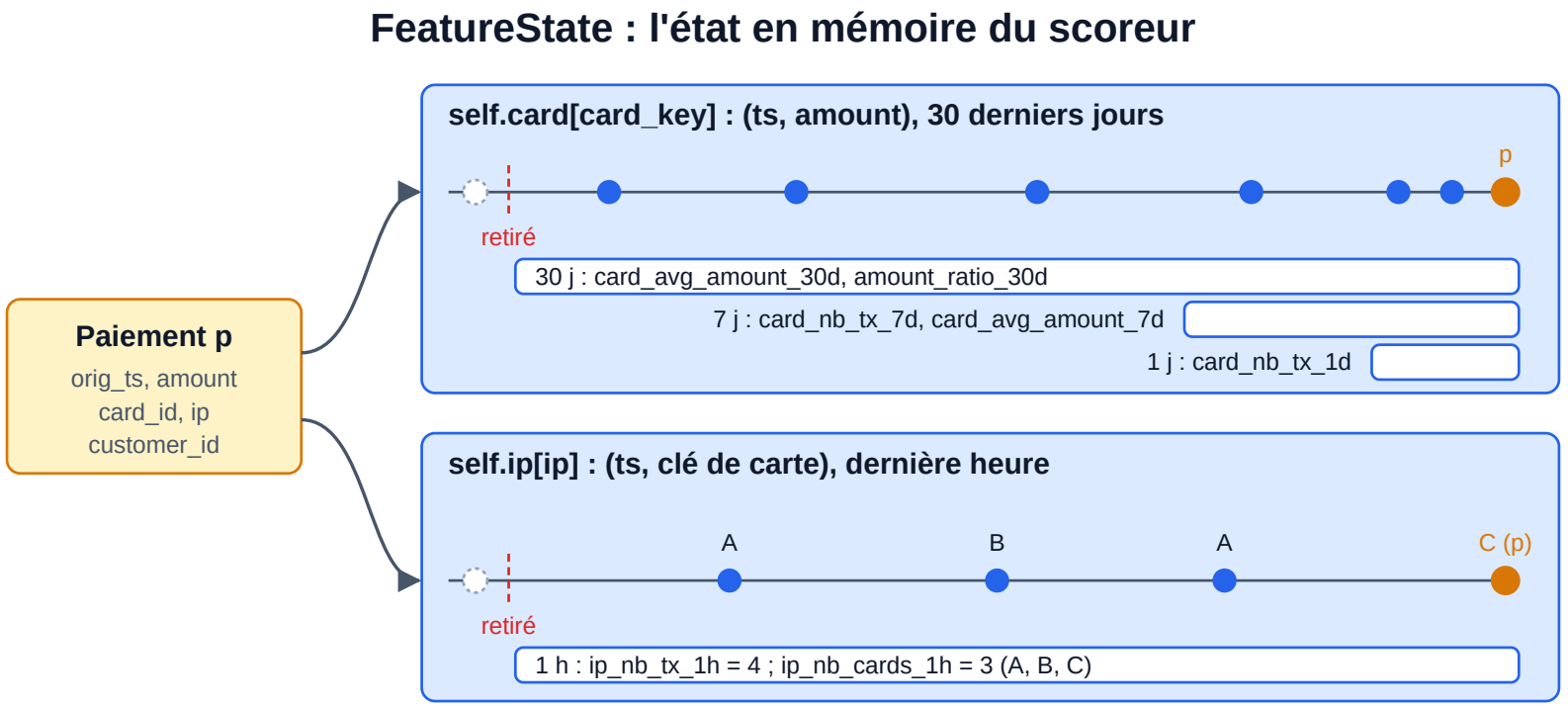
3.A La table des scores

```
bq mk --table $PROJECT:shopflow_rt.scores \
  payment_id:INTEGER,orig_ts:TIMESTAMP,event_ts:TIMESTAMP,amount:FLOAT,status:STRING,score:FLOAT,alert:BOOLEAN,model_version:STRING,mode:STRING,latency_ms:FLOAT,features:STRING
bq show --schema --format=prettyjson $PROJECT:shopflow_rt.scores | head -20
```

3.B Le script `scorer.py`

Le scoreur garde **en mémoire** l'historique des 30 derniers jours de paiements (en temps ShopFlow), refusés compris. À chaque lot, il ajoute les nouveaux paiements à l'historique, puis appelle `features_lib.add_features` sur l'historique des cartes et des IP du lot :

3.B Le script `scorer.py` (suite)



Le paiement courant est inclus dans chaque fenêtre (comme RANGE … CURRENT ROW au TP 2).
Temps = orig_ts ; échelle non proportionnelle.

Schéma : les fenêtres que `features_lib` calcule sur l'historique en mémoire, par carte (1, 7 et 30 jours) et par IP (1 heure) ; les paiements de plus de 30 jours sont retirés de l'historique.

3.B Le script `scorer.py` (suite)

Le script, commenté étape par étape :

3.B Le script `scorer.py` (suite)

```
cat > ~/tp4/scorer.py <<'EOF'
import hashlib, json, os, sys, time
from collections import deque
from datetime import datetime, timedelta, timezone

import joblib
import numpy as np
import pandas as pd
import polars as pl
import requests
from confluent_kafka import Consumer, Producer
from google.cloud import bigquery

import features_lib as fl
from kafka_conf import conf

MODE = sys.argv[1]                    # "api" ou "local"
BINOME, PROJECT = os.environ["BINOME"], os.environ["PROJECT"]
PROJET_ENSEIGNANT = os.environ["PROJET_ENSEIGNANT"]
meta = json.load(open(os.path.expanduser("~/tp5/api/model/meta.json")))
FEATURES = meta["features"]
```

3.B Le script `scorer.py` (suite)

```
# ---------- 0. la librairie de features est-elle exactement celle de l'entraînement ? ----------
lib_hash = hashlib.sha256(open(fl.__file__, "rb").read()).hexdigest()
if lib_hash != meta["features_lib_sha256"]:
    sys.exit("features_lib.py est différent de celui de l'entraînement : reprenez le fichier du volume models")
print("features_lib.py identique à l'entraînement, polars", pl.__version__, flush=True)

# ---------- 1. les clients (pays, date d'inscription), lus une fois dans l'entrepôt du TP 1 ----------
bq = bigquery.Client(project=PROJECT)
cust = bq.query(f"SELECT customer_id, country, signup_date "
                f"FROM `{PROJET_ENSEIGNANT}.shopflow_sf1.customers`").to_dataframe()
CUSTOMERS = {r.customer_id: (r.country, None if pd.isna(r.signup_date) else r.signup_date)
             for r in cust.itertuples()}
print(f"{len(CUSTOMERS)} clients chargés", flush=True)

# ---------- 2. l'historique en mémoire : 30 jours de paiements (temps ShopFlow), refusés compris ----------
history = pl.DataFrame(schema=fl.SCHEMA)


def to_row(p: dict) -> dict:
    """Un message Kafka -> une ligne au format attendu par features_lib."""
    ts = datetime.fromisoformat(p["orig_ts"])        # orig_ts : l'heure d'origine, pas l'heure du rejeu
    if ts.tzinfo is not None:
```

3.B Le script `scorer.py` (suite)

```
        ts = ts.astimezone(timezone.utc).replace(tzinfo=None)
    country, signup = CUSTOMERS.get(p["customer_id"], (None, None))
    return {"payment_id": p["payment_id"], "payment_ts": ts, "amount": float(p["amount"]),
            "ip": p.get("ip"), "customer_id": p["customer_id"], "card_id": p.get("card_id"),
            "shipping_country": p.get("shipping_country"), "customer_country": country,
            "signup_date": signup}


def features_of_batch(batch: list[dict]) -> dict:
    """Ajoute le lot à l'historique, puis calcule ses features avec features_lib (le même code qu'à l'entraînement)."""
    global history
    new = pl.DataFrame([to_row(p) for p in batch], schema=fl.SCHEMA)
    history = pl.concat([history, new])
    history = history.filter(pl.col("payment_ts") >= history["payment_ts"].max() - timedelta(days=31))
    # on ne recalcule que sur l'historique des cartes et des IP du lot : suffisant pour leurs fenêtres
    card_key = pl.coalesce("card_id", -pl.col("payment_id"))
    keys = new.select(card_key.alias("k"))["k"].to_list()
    ips = [ip for ip in new["ip"].to_list() if ip is not None]
    ctx = history.filter(card_key.is_in(keys) | pl.col("ip").is_in(ips))
    out = fl.add_features(ctx).filter(pl.col("payment_id").is_in(new["payment_id"].to_list()))
    return {r["payment_id"]: {f: r[f] for f in FEATURES} for r in out.iter_rows(named=True)}
```

3.B Le script `scorer.py` (suite)

```
local_model = joblib.load(os.path.expanduser("~/tp5/api/model/fraud_model.joblib")) if MODE == "local" else None
http = requests.Session()


# ---------- 3. le score : appel à l'API, ou modèle dans le processus ----------
def score(payment_id: int, feats: dict) -> tuple[float, float]:
    t = time.perf_counter()
    if MODE == "api":
        r = http.post(os.environ["API_URL"] + "/predict", json={"payment_id": payment_id, **feats}, timeout=10)
        s = r.json()["score"]
    else:
        s = float(local_model.predict_proba(pd.DataFrame([feats])[FEATURES])[0, 1])
    return s, (time.perf_counter() - t) * 1000          # score, latence en ms


# ---------- 4. la boucle : lire un lot, dédoublonner, calculer, scorer, publier, écrire ----------
consumer = Consumer(conf(**{"group.id": f"{BINOME}-scorer-{MODE}", "auto.offset.reset": "latest"}))
consumer.subscribe(["payments"])
producer = Producer(conf())
ALERT_TOPIC = f"alerts-{BINOME}"
```

3.B Le script `scorer.py` (suite)

```
seen = deque(maxlen=200_000); seen_set = set()      # les 200 000 derniers payment_id vus
rows, latencies, feat_ms, last_flush = [], [], [], time.time()
print(f"scoreur démarré, mode {MODE}, modèle version {meta['model_version']}", flush=True)
try:
    while True:
        batch = []
        for msg in consumer.consume(num_messages=500, timeout=1.0):   # jusqu'à 500 messages ou 1 seconde
            if msg.error():
                continue
            p = json.loads(msg.value())
            # doublon (le flux en contient volontairement) : on l'ignore
            if p["payment_id"] in seen_set:
                continue
            if len(seen) == seen.maxlen:
                seen_set.discard(seen[0])
            seen.append(p["payment_id"])
            seen_set.add(p["payment_id"])
            batch.append(p)

        if batch:
            t = time.perf_counter()
            feats_by_id = features_of_batch(batch)
```

3.B Le script `scorer.py` (suite)

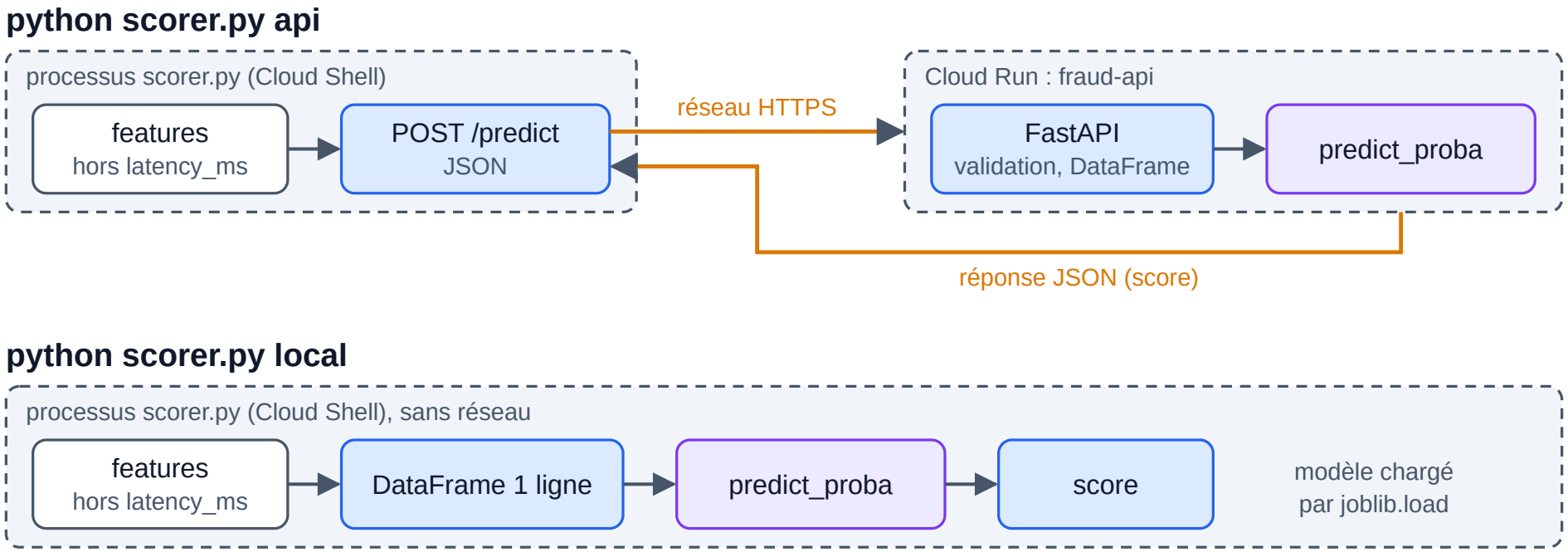
```
            feat_ms.append((time.perf_counter() - t) * 1000)
            for p in batch:
                feats = feats_by_id[p["payment_id"]]
                s, ms = score(p["payment_id"], feats)
                latencies.append(ms)
                alert = s >= meta["threshold"]
                if alert:
                    producer.produce(ALERT_TOPIC, key=str(p["payment_id"]),
                                     value=json.dumps({"payment_id": p["payment_id"], "score": s,
                                                       "model_version": meta["model_version"]}))
                rows.append({"payment_id": p["payment_id"], "orig_ts": p["orig_ts"], "event_ts": p["event_ts"],
                             "amount": p["amount"], "status": p["status"], "score": s, "alert": bool(alert),
                             "model_version": meta["model_version"], "mode": MODE, "latency_ms": ms,
                             "features": json.dumps(feats)})
        producer.poll(0)
        # toutes les 200 lignes ou toutes les 5 secondes : écriture dans BigQuery et bilan
        if rows and (len(rows) >= 200 or time.time() - last_flush > 5):
            errors = bq.insert_rows_json(f"{PROJECT}.shopflow_rt.scores", rows)
            if errors:
                print("BigQuery :", errors[:3], flush=True)
            n_alerts = sum(r["alert"] for r in rows)
            print(f"{len(rows)} scores ({n_alerts} alertes) ; historique {history.height} paiements ; "
```

3.B Le script `scorer.py` (suite)

```
                f"features {np.mean(feat_ms[-50:]):.0f} ms par lot ; "
                f"latence du score p50 {np.percentile(latencies, 50):.1f} ms, p99 {np.percentile(latencies, 99):.1f} ms",
                flush=True)
            rows, last_flush = [], time.time()
            latencies = latencies[-5000:]
except KeyboardInterrupt:
    pass
finally:
    consumer.close()
    producer.flush(10)
EOF
```

3.C Lancer les deux modes

Ce que mesure latency_ms dans chaque mode



latency_ms mesure l'appel à score() : tout ce qui suit le calcul des features.

Schéma : le chemin suivi par un paiement dans chacun des deux modes ; `latency_ms` mesure l'appel à `score()`, après le calcul des features.

3.C Lancer les deux modes (suite)

1. **Onglet 1** (commande d'entrée, puis) :

```
date -u +%H:%M:%S; python scorer.py api 2>&1 | tee scorer_api.log
```

2. **Onglet 2** (bouton `+`, commande d'entrée, puis) :

```
date -u +%H:%M:%S; python scorer.py local 2>&1 | tee scorer_local.log
```

3.C Lancer les deux modes (suite)

Les deux scoreurs sont dans deux groupes de consommateurs différents (`…-scorer-api` et `…-scorer-local`) : chacun reçoit tout le flux. `tee` garde aussi leur sortie dans un fichier, pour la relire plus tard (`tail scorer_local.log`).

> **Ne tapez plus rien dans les onglets 1 et 2** tant que les scoreurs tournent. Une commande collée dans cet onglet part dans l'entrée du scoreur, et un `Ctrl+C` l'arrête. Toutes les autres commandes se tapent dans l'onglet 3.

3. Laissez-les tourner **10 minutes**. Toutes les 5 secondes environ, chacun affiche une ligne `N scores (A alertes) ; historique … paiements ; features … ms par lot ; latence du score p50 … ms, p99 … ms`. Notez, pour chaque mode, la taille de l'historique, le temps de calcul des features par lot, et la latence p50 et p99, au bout de 2 minutes puis au bout de 10 minutes. Le tout premier lot est lent (plusieurs centaines de millisecondes pour les features, et un p99 élevé en mode `local`) : c'est la mise en route de Polars et de scikit-learn.

3.C Lancer les deux modes (suite)

Onglet 3. Lire les alertes publiées dans votre topic. Les deux scoreurs publient dans le **même** topic `alerts-<binome>` : chaque alerte y apparaît donc deux fois, une par mode, avec le même score.

3.C Lancer les deux modes (suite)

```
cd ~/tp4 && source bin/activate && source env.sh
gcloud auth list 2>&1 | grep -q ACTIVE || echo "Pas d'identifiants dans cet onglet : rechargez la page Cloud Shell (les onglets sont conservés)"
cat > ~/tp4/read_alerts.py <<'EOF'
import os
from confluent_kafka import Consumer
from kafka_conf import conf

c = Consumer(conf(**{"group.id": os.environ["BINOME"] + "-alertes", "auto.offset.reset": "earliest"}))
c.subscribe(["alerts-" + os.environ["BINOME"]])
n = 0
for _ in range(30):
    m = c.poll(2.0)
    if m is not None and not m.error():
        print(m.value().decode())
        n += 1
print(n, "alertes lues")
c.close()
EOF
python read_alerts.py
```

3.C Lancer les deux modes (suite)

Vérifiez que les scores arrivent dans BigQuery :

```
bq query --use_legacy_sql=false \
  'SELECT mode, COUNT(*) AS n, COUNTIF(alert) AS n_alertes,
          APPROX_QUANTILES(latency_ms, 100)[OFFSET(50)] AS p50_ms,
          APPROX_QUANTILES(latency_ms, 100)[OFFSET(99)] AS p99_ms
   FROM shopflow_rt.scores GROUP BY mode'
```

3.C Lancer les deux modes (suite)

❓ Q5. Comparez les latences p50 et p99 des deux modes. D'où vient la différence ? Quel mode choisiriez-vous pour autoriser les paiements de ShopFlow, et pourquoi le mode API reste-t-il intéressant ?

Onglet 3. Mesurer le démarrage à froid de l'état. Le script `verif_etat.py` recalcule dans l'entrepôt du TP 1, avec **tout** l'historique, le vrai nombre de paiements de chaque carte sur 1, 7 et 30 jours, pour les paiements que votre scoreur `local` a scorés. Il les compare ensuite, jour ShopFlow par jour ShopFlow, aux valeurs que le scoreur a calculées avec son historique parti de zéro. Lancez-le après 10 minutes, puis de nouveau à la fin de la partie 4 :

3.C Lancer les deux modes (suite)

```
cat > ~/tp4/verif_etat.py <<'EOF'
import os
import pandas as pd
from google.cloud import bigquery

PE = os.environ["PROJET_ENSEIGNANT"]
SQL = f"""
WITH p AS (
  SELECT payment_id, card_id, payment_ts FROM `{PE}.shopflow_sf1.payments`
  WHERE payment_ts BETWEEN '2025-09-25' AND '2025-12-01'
),
vraies AS (
  SELECT payment_id,
    IF(card_id IS NULL, 1, COUNT(*) OVER (PARTITION BY card_id ORDER BY UNIX_SECONDS(payment_ts)
        RANGE BETWEEN 86400 PRECEDING AND CURRENT ROW)) AS vrai_1d,
    IF(card_id IS NULL, 1, COUNT(*) OVER (PARTITION BY card_id ORDER BY UNIX_SECONDS(payment_ts)
        RANGE BETWEEN 604800 PRECEDING AND CURRENT ROW)) AS vrai_7d,
    IF(card_id IS NULL, 1, COUNT(*) OVER (PARTITION BY card_id ORDER BY UNIX_SECONDS(payment_ts)
        RANGE BETWEEN 2592000 PRECEDING AND CURRENT ROW)) AS vrai_30d
  FROM p
)
SELECT DATE(s.orig_ts) AS jour_shopflow, COUNT(*) AS n,
```

3.C Lancer les deux modes (suite)

```
       ROUND(AVG(CAST(JSON_VALUE(s.features, '$.card_nb_tx_1d') AS FLOAT64)), 3) AS scoreur_1d,
       ROUND(AVG(vrai_1d), 3) AS vrai_1d,
       ROUND(AVG(CAST(JSON_VALUE(s.features, '$.card_nb_tx_7d') AS FLOAT64)), 3) AS scoreur_7d,
       ROUND(AVG(vrai_7d), 3) AS vrai_7d,
       ROUND(AVG(vrai_30d), 3) AS vrai_30d
FROM shopflow_rt.scores AS s JOIN vraies USING (payment_id)
WHERE s.mode = 'local'
GROUP BY jour_shopflow ORDER BY jour_shopflow"""
print(bigquery.Client(project=os.environ["PROJECT"]).query(SQL).to_dataframe().to_string(index=False))
EOF
python verif_etat.py 2>&1 | grep -v -i warn
```

Si vous avez lancé le scoreur `local` plusieurs fois, les premiers jours mélangent plusieurs démarrages : ne regardez que les jours scorés depuis le dernier lancement.

3.C Lancer les deux modes (suite)

❓ Q6. L'historique du scoreur démarre vide. Quelles features sont fausses pendant les premières minutes, et pendant combien de temps (en temps ShopFlow et en temps réel) ? Appuyez-vous sur `verif_etat.py` : au bout de combien de jours ShopFlow la fenêtre d'un jour devient-elle juste ? Où en est celle de 7 jours ? Et celle de 30 jours, que vaudrait-elle ? Proposez deux façons de corriger ce démarrage à froid de l'état.

❓ Q7. Le scoreur garde **tous** les paiements dans son historique, refusés compris. Est-ce le bon choix ? Comparez avec la cellule 2 de la partie 1 et avec le calcul de `gold_payment_features` au TP 2.

Partie 4 : surveiller

4.0 Arrêter le mode API

Dans l'onglet 1, arrêtez le scoreur `api` avec `Ctrl+C`, puis notez l'heure : à partir de maintenant, plus personne n'appelle votre API. **Laissez tourner le scoreur `local`** (onglet 2) pendant toute la partie 4.

```
echo "scoreur api arrêté à $(date -u +%H:%M:%S) UTC"
```

Au moins **15 minutes plus tard** (à la fin de la partie 4), relancez dans l'onglet 1 :

```
echo "maintenant $(date -u +%H:%M:%S) UTC"; python ~/tp5/bench.py; python ~/tp5/bench.py
```

et complétez la Q4. La première requête du premier lancement doit prendre plusieurs secondes, et non plus une centaine de millisecondes.

4.1 Les requêtes de monitoring

Le script `monitoring.py` contient cinq requêtes SQL sur `shopflow_rt.scores`. Lancez-le avec le numéro de la requête voulue :

4.1 Les requêtes de monitoring (suite)

```
cat > ~/tp4/monitoring.py <<'EOF'
import os, sys
import pandas as pd
from google.cloud import bigquery

PROJECT, PE = os.environ["PROJECT"], os.environ["PROJET_ENSEIGNANT"]
pd.set_option("display.width", 200, "display.max_rows", 200)

REQUETES = {
    # 1. volume d'alertes par tranche de 5 minutes réelles et par mode
    "1": """
        SELECT TIMESTAMP_SECONDS(DIV(UNIX_SECONDS(event_ts), 300) * 300) AS tranche, mode,
               COUNT(*) AS n, COUNTIF(alert) AS n_alertes,
               ROUND(SAFE_DIVIDE(COUNTIF(alert), COUNT(*)), 4) AS part_alertes
        FROM shopflow_rt.scores
        GROUP BY tranche, mode ORDER BY tranche, mode""",
    # 2. précision des alertes : on « triche » en lisant les rétrofacturations des données d'origine
    "2": f"""
        SELECT s.alert,
               COUNT(*) AS n,
               COUNTIF(p.chargeback_ts IS NOT NULL) AS n_fraudes,
               ROUND(SAFE_DIVIDE(COUNTIF(p.chargeback_ts IS NOT NULL), COUNT(*)), 4) AS taux_fraude
```

4.1 Les requêtes de monitoring (suite)

```
    FROM shopflow_rt.scores AS s
    JOIN `{PE}.shopflow_sf1.payments` AS p USING (payment_id)
    WHERE s.mode = 'local' AND s.status = 'authorized'
    GROUP BY s.alert""",
# 3. rappel : parmi les fraudes autorisées, combien ont déclenché une alerte ?
"3": f"""
    SELECT COUNT(*) AS fraudes_autorisees, COUNTIF(s.alert) AS detectees,
           ROUND(SAFE_DIVIDE(COUNTIF(s.alert), COUNT(*)), 4) AS rappel
    FROM shopflow_rt.scores AS s
    JOIN `{PE}.shopflow_sf1.payments` AS p USING (payment_id)
    WHERE s.mode = 'local' AND s.status = 'authorized' AND p.chargeback_ts IS NOT NULL""",
# 4. démarrage à froid de l'état : moyenne de trois features par tranche de 10 minutes réelles
"4": """
    SELECT TIMESTAMP_SECONDS(DIV(UNIX_SECONDS(event_ts), 600) * 600) AS tranche,
           COUNT(*) AS n,
           ROUND(AVG(CAST(JSON_VALUE(features, '$.card_nb_tx_7d') AS FLOAT64)), 3) AS moy_card_nb_tx_7d,
           ROUND(AVG(CAST(JSON_VALUE(features, '$.card_nb_tx_1d') AS FLOAT64)), 3) AS moy_card_nb_tx_1d,
           ROUND(AVG(CAST(JSON_VALUE(features, '$.ip_nb_cards_1h') AS FLOAT64)), 3) AS moy_ip_nb_cards_1h
    FROM shopflow_rt.scores WHERE mode = 'local'
    GROUP BY tranche ORDER BY tranche""",
# 5. alertes par scénario de fraude (vérité terrain du générateur) et par statut du paiement
"5": f"""
```

4.1 Les requêtes de monitoring (suite)

```
        SELECT IFNULL(p.fraud_scenario, '(légitime)') AS scenario, s.status,
               COUNT(*) AS n, COUNTIF(s.alert) AS alertes,
               COUNTIF(p.chargeback_ts IS NOT NULL) AS retrofacturees,
               ROUND(AVG(s.score), 4) AS score_moyen
        FROM shopflow_rt.scores AS s
        JOIN `{PE}.shopflow_sf1.payments` AS p USING (payment_id)
        WHERE s.mode = 'local'
        GROUP BY scenario, s.status ORDER BY alertes DESC, n DESC""",
}

client = bigquery.Client(project=PROJECT)
for k in (sys.argv[1:] or REQUETES):
    print(f"\n--- requête {k} ---")
    print(client.query(REQUETES[k]).to_dataframe().to_string(index=False))
EOF
```

4.1 Les requêtes de monitoring (suite)

Dans l'onglet 3 (ajoutez `2>&1 | grep -v -i warn` au bout de chaque commande pour masquer l'avertissement « BigQuery Storage module not found », sans conséquence) :

1. **Volume d'alertes** : `python monitoring.py 1`. La part d'alertes est-elle stable d'une tranche à l'autre ? Est-elle proche de 1 % (le seuil choisi en partie 1) ?
2. **Précision des alertes** : `python monitoring.py 2`. Dans la réalité, les labels (rétrofacturations) arrivent 7 à 60 jours plus tard ; ici, on les lit directement dans les données d'origine. La ligne `alert = true` donne la précision de vos alertes.
3. **Fraudes manquées** : `python monitoring.py 3`.
4. **Démarrage à froid de l'état** : `python monitoring.py 4`. Regardez comment les moyennes évoluent depuis le lancement du scoreur. Une moyenne de `ip_nb_cards_1h` qui grimpe nettement au-dessus de 1 sur une tranche signale un **test de cartes** en cours.

4.1 Les requêtes de monitoring (suite)

5. **Alertes par scénario** : `python monitoring.py 5`. Pour chaque scénario de fraude du générateur (`fraud_scenario`, vide pour un paiement légitime) et chaque statut, le nombre de paiements scorés, d'alertes et de rétrofacturations, et le score moyen.
6. Relancez `python verif_etat.py` (partie 3) pour compléter la Q6.

❓ Q8. Quelle précision ont vos alertes ? Combien de fraudes autorisées ont échappé au modèle ? Comparez avec ce que vous attendiez d'après la `precision_top10` du test (partie 1). Avec la requête 5 : quels scénarios de fraude sont présents, lesquels déclenchent les alertes, et d'où viennent les alertes sur des paiements qui ne sont pas rétrofacturés ? Le seuil (1 % des paiements du test) donne-t-il 1 % d'alertes en novembre ?

4.2 La dérive du montant (PSI)

Le script `psi.py` compare les montants des 2 000 derniers paiements scorés aux déciles des montants d'entraînement (`amount_edges` de `meta.json`). Par construction, chaque décile contient 10 % des paiements d'entraînement ; le PSI mesure à quel point les proportions d'aujourd'hui s'en écartent :

4.2 La dérive du montant (PSI) (suite)

```
cat > ~/tp4/psi.py <<'EOF'
import json, os
import numpy as np
from google.cloud import bigquery


def psi_amount(client) -> tuple[float, np.ndarray]:
    meta = json.load(open(os.path.expanduser("~/tp5/api/model/meta.json")))
    edges = np.array(meta["amount_edges"]); edges[0], edges[-1] = -np.inf, np.inf
    amounts = client.query("""
        SELECT amount FROM shopflow_rt.scores WHERE mode = 'local' ORDER BY event_ts DESC LIMIT 2000
    """).to_dataframe().amount.to_numpy()
    q, _ = np.histogram(amounts, bins=edges)          # nombre de paiements dans chaque décile
    q = np.maximum(q / q.sum(), 1e-4)                 # proportions, sans zéro (le log n'aime pas 0)
    p = np.full(len(q), 0.1)                          # proportions à l'entraînement : 10 % par décile
    return float(np.sum((q - p) * np.log(q / p))), q


if __name__ == "__main__":
    psi, q = psi_amount(bigquery.Client(project=os.environ["PROJECT"]))
    print("proportions par décile :", np.round(q, 3))
    print(f"PSI montant = {psi:.3f}")
```

4.2 La dérive du montant (PSI) (suite)

```
EOF
python psi.py
```

Repère usuel : PSI < 0,1, pas de changement notable ; entre 0,1 et 0,25, changement modéré ; > 0,25, changement important.

❓ Q9. Valeur du PSI ? Le mois rejoué (novembre, Black Friday) diffère-t-il de l'entraînement ? Rapprochez ce résultat du volume de paiements par tranche (requête 1). Que décideriez-vous ?

4.3 Le tableau de bord

Le tableau de bord **Streamlit** du TP 5 se rafraîchit toutes les 30 secondes. Il affiche le nombre d'alertes par tranche de 5 minutes et par mode, la latence p50 et p99 de chaque mode, et le PSI du montant :

4.3 Le tableau de bord (suite)

```
cat > ~/tp4/dashboard_tp5.py <<'EOF'
import os

import streamlit as st
from google.cloud import bigquery
from psi import psi_amount

client = bigquery.Client(project=os.environ["PROJECT"])
st.set_page_config(page_title="ShopFlow fraude", layout="wide")
st.title("ShopFlow : scoreur de fraude")


@st.fragment(run_every="30s")
def live():
    df = client.query("""
        SELECT TIMESTAMP_SECONDS(DIV(UNIX_SECONDS(event_ts), 300) * 300) AS tranche, mode,
               COUNTIF(alert) AS n_alertes
        FROM shopflow_rt.scores
        GROUP BY tranche, mode
    """).to_dataframe()
    if df.empty:
        st.info("Pas encore de scores dans shopflow_rt.scores : lancez le scoreur.")
```

4.3 Le tableau de bord (suite)

```
        return

    psi, _ = psi_amount(client)
    lat = client.query("""
        SELECT mode, APPROX_QUANTILES(latency_ms, 100)[OFFSET(50)] AS p50,
               APPROX_QUANTILES(latency_ms, 100)[OFFSET(99)] AS p99
        FROM shopflow_rt.scores
        WHERE event_ts > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 MINUTE)
        GROUP BY mode
    """).to_dataframe()

    cols = st.columns(1 + len(lat))
    cols[0].metric("PSI du montant (2 000 derniers paiements)", f"{psi:.3f}")
    for c, r in zip(cols[1:], lat.itertuples()):
        c.metric(f"Latence {r.mode}, p50 / p99 (30 min)", f"{r.p50:.1f} / {r.p99:.1f} ms")

    st.subheader("Alertes par tranche de 5 minutes")
    st.line_chart(df.pivot_table(index="tranche", columns="mode", values="n_alertes", aggfunc="sum"))


live()
EOF
```

4.3 Le tableau de bord (suite)

```
pkill -f streamlit    # arrête le tableau de bord du TP 4 s'il tourne encore
nohup streamlit run dashboard_tp5.py --server.port 8080 --server.enableCORS false --server.enableXsrfProtection false > streamlit.log 2>&1 &
sleep 6; cat streamlit.log
```

Le tableau de bord tourne en arrière-plan (`nohup … &`) : l'onglet 3 reste libre pour les autres commandes. Ouvrez-le avec le bouton **Aperçu sur le Web > Prévisualiser sur le port 8080** de Cloud Shell. Les heures de l'axe sont celles de votre navigateur, pas l'UTC. Chaque rafraîchissement lance trois requêtes BigQuery : fermez l'onglet du tableau de bord quand vous ne le regardez plus.

4.4 Les métriques du service Cloud Run

1. Console Google Cloud, barre de recherche : **Cloud Run**, puis cliquez sur le service `fraud-api`.
2. Onglet **Métriques**. Choisissez la période *1 heure* en haut à droite.
3. Relevez les graphiques **Nombre de requêtes**, **Latences des requêtes** (courbes p50, p95, p99) et **Nombre d'instances de conteneur** (descendez dans la page). Regardez en particulier ce qui se passe au moment où vous avez arrêté le scoreur `api` (étape 4.0), puis au moment du `bench.py` à froid. Le démarrage à froid de plusieurs secondes apparaît-il dans le graphique des latences ? Comparez avec la mesure de `bench.py` : que mesure chacun ?

Partie 5 : bilan d'architecture

❓ Q10. Dessinez le **schéma complet** de ShopFlow tel que vous l'avez construit en 5 séances (sources, Kafka, lac, entrepôt, traitements, modèle, service, monitoring). Partez du schéma de la vue d'ensemble, corrigez-le si votre réalisation diffère, et indiquez pour chaque flèche : batch ou flux, et la garantie de livraison.

❓ Q11. Estimez le **coût mensuel** en production avec le simulateur de prix Google Cloud, pour un ShopFlow 10 fois plus gros que le jeu SF1. Écrivez vos hypothèses.

❓ Q12. Quels sont les **trois risques** principaux de ce système en production, et que mettriez-vous en place contre chacun ? Aidez-vous de la grille *ML Test Score* du cours.

Fin de séance : tout arrêter et supprimer

1. Arrêtez le scoreur `local` (onglet 2) et le tableau de bord avec `Ctrl+C`, ou d'un coup :

```
pkill -f scorer.py; pkill -f streamlit
```

2. Supprimez le service Cloud Run et le dépôt d'images créé par `--source .` :

```
gcloud run services delete fraud-api --region=europe-west1
gcloud artifacts repositories delete cloud-run-source-deploy --location=europe-west1
```

3. Quand vous avez fini d'exploiter vos résultats, supprimez aussi les données. Attention : ces deux commandes effacent aussi les résultats du TP 4 (tables `*_by_minute` et `ip_alerts`, bronze et checkpoints du bucket) :

Fin de séance : tout arrêter et supprimer (suite)

```
bq rm -r -f $PROJECT:shopflow_rt
gcloud storage rm -r $BUCKET
```

Si quelque chose bloque

Symptôme	Piste
Cellule 2 : `AssertionError` dans `_cartes_par_ip_1h`	Votre `features_lib.py` n'a pas la ligne `.sort("ip", "payment_ts", maintain_order=True)` : recopiez le fichier de l'étape 1.0, puis **Run all** (la cellule 1 redémarre Python, sinon l'ancien module reste en mémoire)
`toPandas()` lent ou en échec mémoire (cellule 2)	Relancer la cellule ; si cela persiste, prévenir l'enseignant (sous-échantillonner les paiements légitimes de 2024)
`log_model` : argument `name` inconnu	La cellule 1 (`%pip install "mlflow>=3"`) n'a pas été exécutée, ou le notebook n'a pas redémarré : la relancer, puis toutes les cellules suivantes
Pas de bouton **Download** sur le volume	Ouvrir le fichier dans *Catalog* > *Volumes*, puis le menu `⋮` en haut à droite
`pip install -r requirements.txt` échoue dans Cloud Shell	La version de scikit-learn ou numpy n'existe pas pour le Python de Cloud Shell : prévenir l'enseignant
Erreur de chargement du modèle (`joblib.load`)	Versions de scikit-learn ou numpy différentes de l'entraînement : vérifier `requirements.txt` contre `meta.json` (étape 2.2)
`gcloud run deploy` : `PERMISSION_DENIED` pendant la construction	Le rôle `roles/run.builder` (étape 2.4) n'est pas encore appliqué : attendre 1 à 2 minutes et relancer
`--allow-unauthenticated` refusé	Contrainte d'organisation : voir l'enseignant
`401` ou `invalid_grant` sur BigQuery depuis	Relancer `gcloud auth application-default login` (TP 4, étape 1 bis)

Si quelque chose bloque (suite)

Symptôme	Piste
« Your session was transferred to another browser tab »	Cloud Shell est ouvert ailleurs (autre navigateur, autre fenêtre) : fermez l'autre onglet, puis **Reconnect**
`MANQUANT : …` à l'étape 2.1	L'import n'a pas abouti, ou a été fait dans un Cloud Shell ouvert avec un autre compte Google : refaites-le depuis l'onglet du bon compte
Étape 2.3 : les `curl` n'affichent rien	L'API n'a pas démarré en 30 s : `cat uvicorn.log` ; relancer le bloc
Le scoreur s'arrête : « features_lib.py est différent de celui de l'entraînement »	Le fichier `~/tp4/features_lib.py` a été modifié ou ne vient pas du volume `models` : le retélécharger (partie 1.C)
`features_lib.py` modifié, mais la cellule 2 utilise encore l'ancienne version	Python garde le module en mémoire : relancer la cellule 1 (`restartPython`), ou **Run all**
`ModuleNotFoundError: features_lib` dans Databricks	Le fichier `features_lib.py` n'est pas dans le même dossier que le notebook `05_train` (étape 1.0)
`KeyError: 'PROJET_ENSEIGNANT'` ou `'API_URL'`	La commande d'entrée de l'onglet n'a pas été tapée, ou l'étape 2.0 / 2.4 n'a pas ajouté la variable à `env.sh`
`insert_rows_json` : table introuvable	Créer `shopflow_rt.scores` (étape 3.A) ; attendre quelques secondes après la création
	Le rejeu de l'enseignant est peut-être terminé (il dure 6 h) : lancer `python`

Si quelque chose bloque (suite)

Symptôme	Piste
`bq query` : `RecursionError`	La requête commence par un commentaire `--`, que `bq` prend pour une option : retirer le commentaire, ou passer par Python comme `verif_etat.py`
Aucune alerte publiée	Normal pendant les premières minutes : la part d'alertes est bien inférieure à 1 % au début du rejeu (nuit du 1er novembre, état du scoreur incomplet). Si rien après 10 minutes, prévenir l'enseignant
Chaque alerte apparaît deux fois dans `read_alerts.py`	Normal : les scoreurs `api` et `local` publient dans le même topic
`TopicAuthorizationException` sur les alertes	Le topic doit commencer par `alerts-` : vérifier `echo $BINOME`